



UNIVERSIDADE FEDERAL DO CEARÁ
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE ENGENHARIA DA COMPUTAÇÃO
PROCESSAMENTO DE IMAGENS E APLICAÇÕES DISTRIBUÍDAS
SEMESTRE 2026.1

Sistema de Reconhecimento Facial Distribuído

Augusto Ferreira de Freitas - 472144

Arthur Bernardo Oliveira da Costa - 563535

Gabriel Miranda dos Santos - 497314

João Lucas Oliveira Mota - 509597

João Victor Leandro Nunes - 414957

Sumário

1	Introdução	3
2	Arquitetura Distribuída e Protocolos de Comunicação	4
2.1	Topologia da Rede e Desacoplamento de Nós	5
2.2	Validação Experimental da Infraestrutura Distribuída	5
2.2.1	Mecanismo de Consulta por similaridade vetorial	6
2.3	Implementação com a leitura de Rostos em Tempo Real	7
2.3.1	AWS	8
2.3.2	Qdrant Cloud	8
2.4	Amazon S3 (Simple Storage Service) e Amazon DynamoDB	9
3	Nó Cliente	10
3.1	Pipeline de Visão Computacional	10
3.1.1	Descrição das Etapas	11
3.2	Isolamento da Região de Interesse (ROI):	11
3.2.1	Detecção de Faces com YOLOv8	11
3.2.2	Resultados do Treinamento e Validação	12
3.3	Extração e Vetorização de Características Faciais	12
4	Conclusão	14

1 Introdução

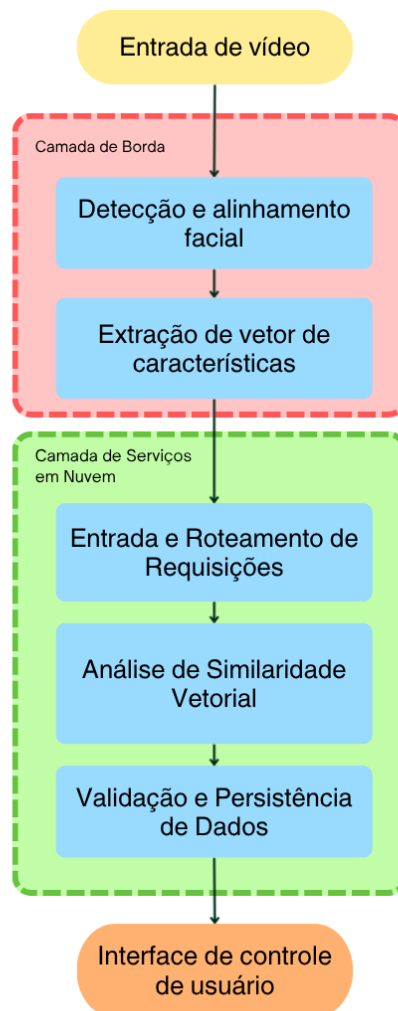
A automação do registro de jornada via reconhecimento facial enfrenta o desafio de substituir crachás físicos por métodos mais seguros e práticos, evitando fraudes e falhas operacionais. Este projeto propõe a implementação de um sistema de ponto eletrônico baseado em uma arquitetura distribuída, que utiliza o processamento local na borda para a captura e filtragem de imagens, delegando a validação inteligente a serviços computacionais na nuvem.

A solução articula a extração local de características biométricas com a busca em bases de dados vetoriais e a orquestração de eventos em ambiente de nuvem. Ao integrar o armazenamento de evidências digitais com a persistência de regras de negócio em um banco de dados NoSQL gerenciado, o sistema elimina a necessidade de infraestrutura local de alto custo, assegurando escalabilidade, auditoria rigorosa e eficiência na alocação de recursos computacionais.

2 Arquitetura Distribuída e Protocolos de Comunicação

O sistema proposto foi modelado sob o paradigma cliente-servidor fracamente acoplado, dividindo a carga de trabalho entre o processamento de borda (*edge computing*) e o armazenamento especializado em nuvem. Essa separação de responsabilidades buscou otimizar o uso dos recursos computacionais locais e garantir a escalabilidade horizontal do banco de dados de vetores. A disposição estrutural de cada componente e a interconexão dos blocos estão apresentadas na Figura 1.

Figura 1: Fluxo de dados e divisão de componentes entre as camadas de Borda e Serviços em nuvem.



Fonte: Elaborada pelo autor.

Conforme observado no diagrama, o fluxo operacional iniciou no ambiente local com a captura de vídeo pelo dispositivo de entrada e o processamento da imagem para a detecção e o alinhamento facial. Na sequência, foi utilizado um modelo para extração de vetores de características com base no rosto alinhado e realizado o envio para a camada em nuvem.

Ao atingir a camada em nuvem, é realizado o processamento e roteamento das requisições para o serviço de busca vetorial, que retorna o resultado da consulta. O sistema então processa a resposta e, com base no resultado, toma a decisão de liberar ou recusar o acesso, realizando uma última validação e atualizando o histórico de acesso no banco de dados NoSQL.

2.1 Topologia da Rede e Desacoplamento de Nós

A topologia do sistema é composta por dois nós principais que operam de forma geograficamente distribuída:

- **Camada de Borda:** Executada em ambiente computacional local, realizando a captura de vídeo, o pré-processamento de imagens, a extração de características faciais e a comunicação com os serviços em nuvem. Esta camada é responsável por realizar as operações de visão computacional e gerar os vetores de características que serão enviados para o servidor.
- **Camada de Serviços em Nuvem:** Consiste em um cluster gerenciado do *Qdrant Cloud*, hospedado na infraestrutura da *Amazon Web Services* (AWS) na região da América do Sul (*sa-east-1*). Este nó é responsável pela indexação espacial, persistência e processamento de consultas de similaridade vetorial.

A comunicação entre as extremidades ocorre por meio de requisições HTTP utilizando uma interface de programação de aplicações baseada na arquitetura REST (*RESTful API*), implementada sobre as bibliotecas de transporte `httpx` e `httpcore`.

2.2 Validação Experimental da Infraestrutura Distribuída

A validação experimental da infraestrutura distribuída foi conduzida utilizando a base de dados de faces da FEI (*FEI Face Database*) [Thomaz and Giraldi 2006]. Este repositório biométrico é composto por 2.800 imagens coloridas com resolução original de 640×480 pixels,

obtidas a partir de um grupo de 200 indivíduos, apresentando uma divisão equitativa de 100 sujeitos do sexo masculino e 100 do sexo feminino, com idades variando entre 19 e 40 anos. Cada participante possui 14 registros fotográficos capturados contra um fundo branco homogêneo, abrangendo diferentes expressões faciais e rotações de perfil de até cerca de 180 graus, com variações de escala estimadas em 10%.

No modelo de testes aplicado ao sistema distribuído, o conjunto de dados foi segmentado e alocado logicamente para simular um cenário real de identificação remota:

- **Fase de Ingestão:** O subconjunto composto pelas imagens em posição estritamente frontal e com expressão neutra permaneceu no nó cliente para o processo de extração local e alimentação síncrona inicial da coleção hospedada no *Qdrant Cloud*.
- **Fase de Consulta:** As imagens que apresentavam variações de pose, rotação e expressões do mesmo indivíduo foram isoladas em um diretório de testes secundário no nó local. Esses arquivos foram utilizados para alimentar o laço contínuo de requisições externas, validando a capacidade da operação de busca vetorial em correlacionar os vetores de características originais com a versão enviada durante os testes.

2.2.1 Mecanismo de Consulta por similaridade vetorial

A etapa de consulta biométrica e identificação no banco de dados vetorial foi realizada por meio de uma chamada de procedimento remoto utilizando o método unificado `query_points` da API do *Qdrant*. Este método consolida a pesquisa espacial em uma única requisição estruturada, otimizando o tempo de resposta do cluster em nuvem. [Qdrant Team 2024].

O vetor de características composto por 512 dimensões, gerado na borda pelo modelo *Arc-Face*, é encaminhado como o argumento principal da consulta no parâmetro `query`. Configura-se o parâmetro `limit` com valor unitário para estabelecer uma busca do tipo *top – 1*, o que instrui o servidor a retornar apenas o vetor que apresenta a maior proximidade espacial. A requisição também define o parâmetro `with_payload` como verdadeiro, garantindo que o servidor anexe os metadados associados ao registro localizado.

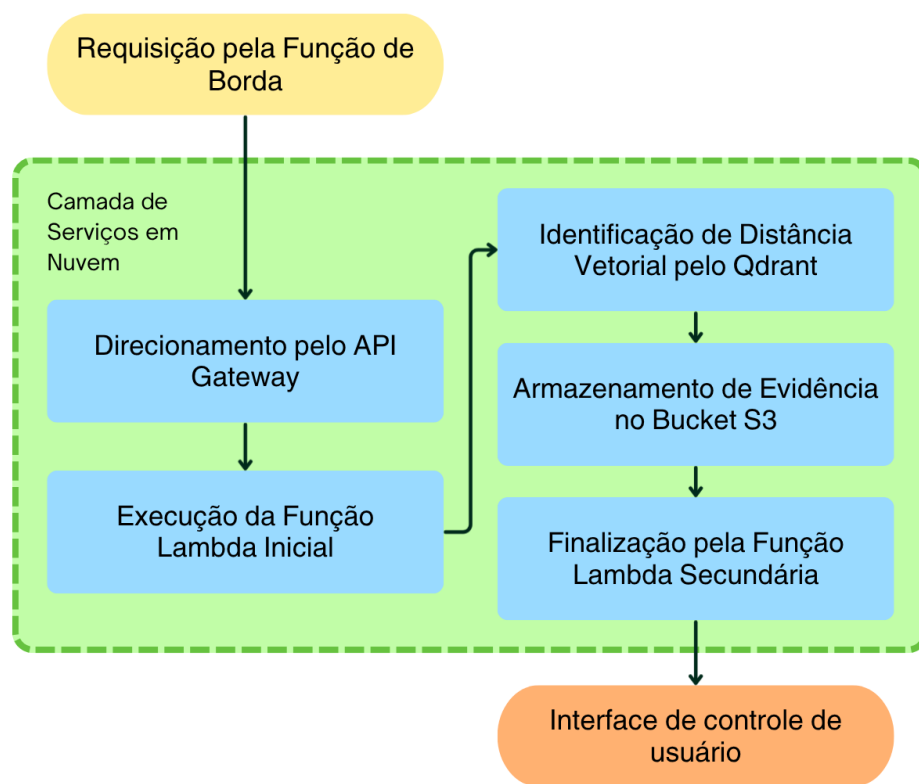
No servidor, o *Qdrant* processa a busca calculando a similaridade de cosseno entre o vetor de entrada e os elementos indexados na coleção. O retorno gerado pelo método é estruturado em uma lista sob o atributo `points`. O sistema analisa o primeiro elemento dessa lista para extrair

o identificador único do ponto, os metadados da imagem original e o escore de similaridade correspondente, o qual serve como métrica de confiança para a validação da identidade.

2.3 Implementação com a leitura de Rostos em Tempo Real

A Figura 2 apresenta um diagrama detalhado da arquitetura da camada de serviços em nuvem, destacando os componentes e fluxos de dados envolvidos nela. O diagrama mostra o passo a passo do processamento das requis

Figura 2: Diagrama detalhado da Camada de Serviços em Nuvem.



Fonte: Elaborada pelo autor.

É importante destacar a escolha de separar o processamento em Nuvem em dois serviços principais, a Amazon AWS e o Qdrant Cloud, cada um com responsabilidades específicas.

A AWS é utilizada para hospedar a infraestrutura de backend, gerenciar a autenticação, orquestrar as requisições e garantir a segurança dos dados, enquanto o Qdrant Cloud é especializado no armazenamento e na consulta eficiente dos vetores de características faciais. Essa divisão permite que cada serviço seja otimizado para suas funções específicas, garantindo uma

operação mais fluida e responsiva do sistema como um todo.

2.3.1 AWS

Para a implementação da camada de serviços em nuvem, o processo começa com o direcionamento das requisições através do Amazon API Gateway, que atua como um ponto de entrada seguro e escalável para as chamadas externas. O API Gateway é configurado para rotear as requisições para uma função Lambda, que é responsável por validar vetor recebido e realizar a consulta ao banco de dados do Qdrant Cloud. Posteriormente, ao receber a resposta do Qdrant, a função Lambda processa os resultados e encaminha a resposta final de volta ao cliente, indicando se o acesso foi liberado ou recusado com base na similaridade dos vetores.

A escolha da AWS para esta camada se deve à sua robustez, escalabilidade e ampla gama de serviços gerenciados que facilitam a implementação de soluções em nuvem, além de garantir a segurança e a confiabilidade necessárias para o processamento de dados sensíveis como os vetores de características faciais. [Amazon Web Services 2026]

2.3.2 Qdrant Cloud

Já no que se refere ao Qdrant Cloud, ele é utilizado como o serviço de banco de dados especializado para armazenar os vetores de características faciais e realizar as consultas de similaridade. O Qdrant é otimizado para lidar com grandes volumes de dados vetoriais e oferece uma API eficiente para indexação e busca, o que o torna ideal para este tipo de aplicação. [Qdrant Team 2024]

Para isso, a Função Lambda da AWS realiza uma consulta do tipo `query_points`, enviando o vetor de características recebido do cliente e solicitando o retorno do ponto mais próximo na coleção do Qdrant. O serviço processa a consulta utilizando algoritmos de busca vetorial e retorna os resultados, que são então interpretados pela função Lambda para determinar a resposta final ao cliente. Essa integração entre a AWS e o Qdrant Cloud permite que o sistema aproveite as vantagens de ambos os serviços, garantindo uma operação eficiente e escalável para o reconhecimento facial distribuído.

2.4 Amazon S3 (Simple Storage Service) e Amazon DynamoDB

Para garantir a rastreabilidade e a segurança do sistema, todas as tentativas de acesso são auditadas utilizando dois serviços complementares da AWS. Assim que a Função Lambda recebe a requisição, a imagem do rosto capturado pelo cliente, onde é enviada codificada no formato Base64, é decodificada e armazenada como evidência visual no Amazon S3 (Simple Storage Service).

Simultaneamente, os metadados da transação (como data, hora, identificação da porta, pontuação de similaridade do Qdrant e o link público para a foto no S3) são persistidos no Amazon DynamoDB, um banco de dados NoSQL altamente escalável. Essa abordagem garante um histórico de logs imutável e de acesso rápido, permitindo consultas e auditorias futuras sem sobrecarregar a camada de processamento vetorial.

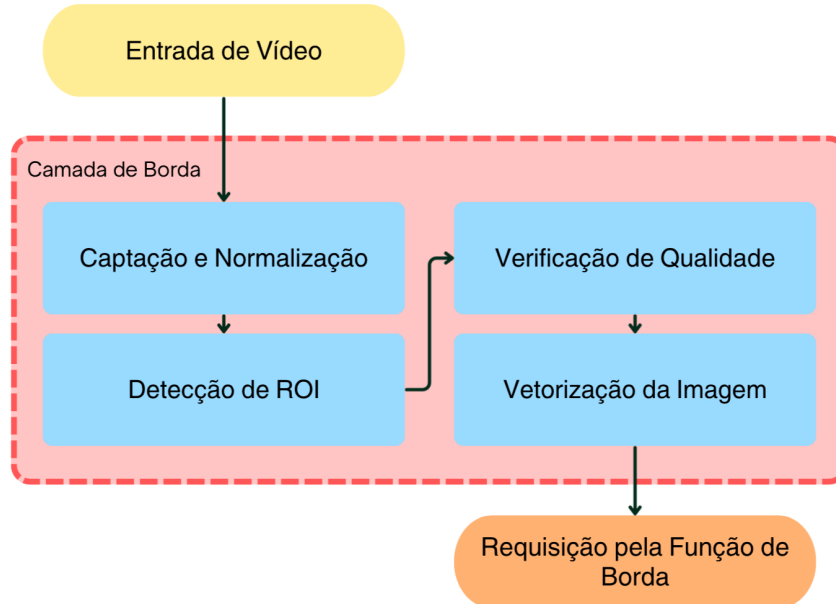
3 Nó Cliente: Pipeline de Processamento de Imagens e Extração de Características

Nesta seção, é detalhado o pipeline de processamento de imagens implementado na camada de borda do sistema. Essa etapa é responsável pela captura contínua dos frames, detecção da face, extração da região de interesse (ROI), verificação da qualidade da amostra facial e geração do vetor de características por meio do modelo ArcFace.

3.1 Pipeline de Visão Computacional

O fluxo de processamento de dados biométricos foi estruturado para assegurar eficiência, qualidade da amostra e confiabilidade na comunicação com os serviços remotos. O processo é composto por cinco etapas principais, representadas na Figura 3:

Figura 3: Funcionamento da pipeline de processamento de imagens e extração de características faciais.



Fonte: Elaborada pelo autor.

3.1.1 Descrição das Etapas

1. **Captação e Normalização:** O sistema inicia com a captura do fluxo de vídeo por meio da interface `cv2.VideoCapture`. Os frames capturados são utilizados como entrada para o modelo de detecção facial, permitindo a análise contínua da cena em tempo real. Essa etapa padroniza o fluxo de entrada do sistema e estabelece a base para as etapas seguintes de detecção, recorte e validação da face.
2. **Detecção e Extração de ROI:** O modelo YOLO atua como detector primário, localizando a face no frame e retornando as coordenadas da caixa delimitadora. A partir dessas coordenadas, o sistema recorta a Região de Interesse (ROI), mantendo apenas a área facial para as etapas seguintes.
3. **Auditoria de Qualidade:** Antes da vetorização, o recorte facial passa por uma verificação de qualidade. São avaliados critérios como área mínima da face, nível médio de brilho, confiança da detecção e nitidez estimada pelo operador Laplaciano. Apenas amostras que atendem aos limiares definidos no sistema prosseguem no *pipeline*, evitando o envio de imagens muito pequenas, escuras, superexpostas ou borradas.
4. **Vetorização (Embedding):** A face validada é convertida pelo modelo *ArcFace* em um vetor numérico de alta dimensionalidade (*embedding*), que sintetiza as características geométricas únicas do indivíduo para posterior comparação matemática.
5. **Comunicação e Validação Remota:** O *embedding*, a identificação da porta e o recorte facial codificado em Base64 são enviados para a infraestrutura em nuvem. Na etapa remota, o vetor é consultado no banco vetorial, a similaridade é avaliada e o sistema retorna ao ponto de origem o status da tentativa de acesso.

3.2 Isolamento da Região de Interesse (ROI):

3.2.1 Detecção de Faces com YOLOv8

A detecção de rostos foi realizada utilizando a arquitetura YOLO (You Only Look Once), uma rede neural convolucional de estágio único que processa a imagem em uma única passagem direta [Redmon et al. 2016]. A implementação baseou-se no framework Ultralytics YOLOv8

[Jocher et al. 2023], utilizado para localizar a face nos frames capturados e retornar as coordenadas da caixa delimitadora. Essa escolha é adequada para aplicações em tempo real, pois permite detectar a região facial com baixa latência antes da etapa de extração de características.

O conjunto de dados, composto pelas 2.800 imagens do *FEI face database*, passou por um processo de normalização e anotação automatizada. O processo utilizou um modelo pré-treinado de detecção facial baseado em Caffé, pré-treinado com arquitetura ResNet-10 SSD, para delimitar as áreas de interesse em cada imagem, garantindo que apenas exemplos com detecções positivas compusessem a base final. O dataset foi, por fim, particionado para o treinamento e a validação da rede, seguindo a divisão de 80% e 20%, respectivamente.

3.2.2 Resultados do Treinamento e Validação

O modelo YOLOv8n foi treinado com *batch size* 4, resolução de 416 pixels e ausência de *data augmentation*. O processo estendeu-se por 87 épocas, com duração total de 4,6 horas e monitoramento contínuo das funções de perda (*box loss*, *cls loss* e *dfl loss*) para garantir a convergência e evitar *overfitting*.

Os resultados confirmaram a eficácia do modelo *best.pt*, que atingiu *mAP@50* de 0,995 e *mAP@50-95* de 0,966, com latência de inferência de 19,5 ms por imagem, confirmando a robustez do sistema para aplicações de reconhecimento facial.

3.3 Extração e Vetorização de Características Faciais

Com a face detectada, recortada e validada, o sistema procede para a extração de características, convertendo a imagem em um vetor numérico de alta dimensionalidade, denominado *embedding facial*. Esse processo é executado localmente por meio da biblioteca DeepFace utilizando o modelo *ArcFace*. A vantagem dessa abordagem é substituir a comparação direta de pixels por uma comparação matemática entre vetores, tornando o processo de reconhecimento mais adequado a variações de iluminação, expressão facial e enquadramento [Serengil and contributors 2026].

A eficácia do *ArcFace* reside na função de perda *Additive Angular Margin Loss*, que otimiza a separação angular entre identidades diferentes. Enquanto modelos tradicionais baseados em *softmax* limitam-se à classificação, o *ArcFace* maximiza a distância entre classes distintas e

minimiza a variação intra-classe no espaço angular, conforme definido pela Equação 1:

$$L = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{s \cos(\theta_{y_i} + m)}}{e^{s \cos(\theta_{y_i} + m)} + \sum_{j=1, j \neq y_i}^n e^{s \cos(\theta_j)}} \quad (1)$$

Nesta expressão, N representa o número de amostras, y_i a classe correta, θ_{y_i} o ângulo entre o vetor de características e o peso da classe, enquanto m e s representam a margem angular e o fator de escala, respectivamente [Deng et al. 2019].

O resultado desse processo é um vetor de *embedding facial* de 512 dimensões, conforme ilustrado na Tabela 1, que encapsula as características faciais de forma compacta e discriminativa, permitindo a comparação eficiente com os vetores previamente cadastrados no banco de dados vetorial.

Tabela 1: Exemplo da estrutura e valores do vetor de embedding gerado pelo modelo ArcFace.

Índice	Valor do Embedding
0	-0.226871639
1	0.036645882
2	0.053239971
3	-0.223227322
⋮	⋮
510	0.07878179
511	0.078781798

4 Conclusão

O projeto confirmou a eficácia de sistemas biométricos distribuídos para controle de acesso, integrando processamento local e nuvem para garantir segurança e performance. A arquitetura centralizada assegura uma gestão auditável e escalável, atendendo aos requisitos de alta confiabilidade corporativa.

No âmbito da visão computacional, o uso do *ArcFace* com processamento local reduziu gargalos de latência e tráfego de rede. Embora o sistema tenha superado as limitações de soluções que dependem de imagens brutas, observou-se a ocorrência de falsos negativos. Tais erros foram atribuídos a variações nos vetores de características, causadas por diferenças na iluminação e na qualidade do sensor da câmera entre a captura em tempo real e o armazenamento no banco de dados.

Para trabalhos futuros, recomenda-se a implementação de detecção de vivacidade (*liveness detection*) para elevar a segurança contra fraudes. Adicionalmente, a otimização dos modelos para hardware de borda e o refinamento dos critérios de qualidade da imagem permitirão escalar a solução para mais dispositivos, mitigando as variações que impactam a precisão do reconhecimento.-

Referências

- [Amazon Web Services 2026] Amazon Web Services (2026). Aws cloud documentation: Serverless computing. Acessado em: 16 de junho de 2026.
- [Deng et al. 2019] Deng, J., Guo, J., Xue, N., and Zafeiriou, S. (2019). Arcface: Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4690–4699.
- [Jocher et al. 2023] Jocher, G., Chaurasia, A., Stoken, A., Borovec, J., NanoCode012, Kwon, Y., Michael, T., Fang, T., Cieplinski, M., and Vlasov, A. (2023). Ultralytics yolov8.
- [Qdrant Team 2024] Qdrant Team (2024). Qdrant documentation - the query API. Acessado em: 3 de junho de 2026.
- [Redmon et al. 2016] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788.
- [Serengil and contributors 2026] Serengil, S. I. and contributors (2026). Deepface: A lightweight face recognition and facial attribute analysis framework for python. <https://github.com/serengil/deepface>. Acesso em: 31 maio 2026.
- [Thomaz and Giraldi 2006] Thomaz, C. E. and Giraldi, G. A. (2006). FEI face database. Acessado em: 2 de junho de 2026.